T E C H N I C A L R E P O R T

Production Monitoring of a Cloud-Deployed Machine Learning Service

Data Drift Detection, Latency Observability and Cost Control on Google Cloud Run

Name	Ritesh Gawade
Roll No.	25P0620014
Course	CSI-6402 Cloud Based AI Deployment
Assignment	Lab Assignment 5 — Model Monitoring for Drift and Latency using Cloud Monitoring Tools
Date	October 2026

1. Abstract

This project implements and deploys a machine-learning prediction service on Google Cloud Platform (GCP) and builds a monitoring workflow around it that tracks both input-data drift and service latency. A scikit-learn Gradient Boosting regression pipeline is served through FastAPI, packaged as a Docker image, stored in Artifact Registry and deployed to Cloud Run in the `asia-south1` region. Each call to the `/predict` endpoint writes the input features and the prediction to a Google Cloud Storage (GCS) bucket, which becomes the "current" production window. A drift endpoint compares the training data (reference window, 469 rows) with the most recent production records (122 rows) using a two-sample Kolmogorov–Smirnov (KS) test at a significance level of 0.05, and stores each result in a CSV report in GCS.

The live drift check flagged both monitored features: `IRRADIATION` (KS = 0.1591, p = 0.0129) and `AMBIENT_TEMPERATURE` (KS = 0.3968, p = 3.44×10^{-14}). A controlled load test of 100 requests at a concurrency of 10 completed with 100 successful and 0 failed requests, with client-observed latency of p50 = 2189.88 ms, p95 = 3277.93 ms and p99 = 3472.61 ms. Cloud Monitoring reported lower server-side percentiles (p50 = 462.02 ms, p95 = 534.06 ms, p99 = 544.92 ms) for the preceding traffic window. Two GCP budget alerts were configured to keep the experiment within a small spending limit. Drift is treated throughout as evidence of a change in input distributions, not as proof of degraded model accuracy.

2. Introduction

A deployed model can keep returning predictions long after the environment it was trained for has changed. Monitoring therefore has to answer two separate questions: do production inputs still resemble the data the model was built on, and does the serving system remain responsive? This project addresses both by combining GCP-managed services (Cloud Run, Artifact Registry, Cloud Storage, Cloud Monitoring and Billing budgets) with a statistical drift test from SciPy.

The case-study model estimates AC power output from irradiation, ambient temperature and time of day. The notebook trains on Plant 1 data and prepares a validation set from Plant 2 data; a helper script then sends Plant 2 rows to the deployed API. This produces a realistic domain-shift experiment, although it should be read as a change of plant rather than a general claim about all real-world production conditions.

2.1 Concepts relevant to this experiment

Concept	Meaning in this project
Data drift	A change in the distribution of model input features between reference and production data. Tested here for `IRRADIATION` and `AMBIENT_TEMPERATURE`.
Label shift	A change in the distribution of target labels while the input–target relationship is stable. Not tested: production logs store predictions, not measured AC power.
Concept drift	A change in the relationship between inputs and the true target. Detecting it requires true outcomes over time, which this project does not log.
Prediction drift	A change in the distribution of model outputs. `predicted_AC_POWER` is logged but not tested by the drift endpoint.
Domain shift	A change in operating environment or population. Plant 1 (training) versus Plant 2 (production generator) is the domain shift used here.

Drift does **not** automatically mean the model has become worse. A KS result concerns input distributions, not prediction error. Judging model quality requires true production outcomes.

3. Problem Statement

Build and deploy a prediction API on Google Cloud, collect prediction-time data, and demonstrate a statistically testable comparison between a reference window and a current production window. In addition, define a reproducible request load and monitor service latency with percentiles (p50, p95, p99) rather than relying on average latency alone, and keep the experiment under cost control.

4. Objectives

1. Prepare solar generation and weather data for regression.
2. Train and evaluate a Gradient Boosting model for `AC_POWER`.
3. Serve predictions through a request-validated FastAPI endpoint.
4. Containerize the service and deploy it through Artifact Registry and Cloud Run.
5. Persist prediction-time features and predictions in Google Cloud Storage.
6. Compare a training reference window with recent production inputs using a statistical test.
7. Measure request latency at p50, p95 and p99 under a stated request load.
8. Set up budget alerts and interpret drift and latency evidence without overstating model degradation.

5. Dataset and Data Preprocessing

5.1 Source and populations

The notebook downloads the Kaggle `solar-power-generation-data` dataset through `kagglehub`. Plant 1 generation and weather-sensor data are used for training and are merged on `DATE_TIME`. Generation records are grouped by timestamp with `AC_POWER` and `DC_POWER`

summed and irradiation and ambient temperature averaged. The data is resampled to hourly records, `DC_POWER` is removed, and records with `IRRADIATION <= 0` are excluded. Time of day is encoded cyclically:

$$\texttt{HOUR_SIN} = \sin(2\pi \cdot \texttt{HOUR} / 24), \quad \texttt{HOUR_COS} = \cos(2\pi \cdot \texttt{HOUR} / 24)$$

The training table drops `DATE_TIME` and the raw `HOUR`, leaving five columns. `data/train.csv` has 469 rows. The Plant 2 validation set is prepared with the same steps; `data/validation.csv` has 513 rows with the same five-column schema.

`scripts/generate_production_data.py` reads the first 60 rows of `data/validation.csv` per execution, assigns hourly timestamps and posts them to `/predict`. The production sample therefore comes from API requests built from validation records, and is separate from the load-test traffic. The drift monitor reads its current window from GCS, not from a local file.

5.2 Final model input and target

Role	Column
Target	`AC_POWER`
Feature	`IRRADIATION`
Feature	`AMBIENT_TEMPERATURE`
Feature	`HOUR_SIN`
Feature	`HOUR_COS`

6. Machine Learning Model

The estimator is `GradientBoostingRegressor(random_state=42)` inside a scikit-learn `Pipeline` with a `StandardScaler`. The pipeline is saved as `models/model_pipeline.joblib` and loaded by `app.py` at start-up. Gradient Boosting was selected over Linear Regression and Random Forest by cross-validated RMSE and R^2 in the notebook.

Evaluation	RMSE	R^2
Held-out 20% split (random_state=42)	523.1009	0.99638
Mean of 5-fold cross-validation	627.6131	0.99369

The 5-fold cross-validation also recorded a standard deviation of 137.27 for RMSE and 0.0029 for R^2. These are saved notebook results, not a fresh retraining run. The unit of `AC_POWER` is not stated in the project files, so no physical unit is assigned to the RMSE values.

7. FastAPI Application

The service in `app.py` exposes six endpoints, all visible in the auto-generated Swagger interface of the deployed service (Figure 1).

Endpoint	Behavior
`GET /`	Returns a short API message.
`GET /model-status`	Reports whether the model loaded successfully.
`POST /predict`	Validates the request, parses `DATE_TIME` (`DD-MM-YYYY HH:MM`), derives cyclical hour features, predicts `AC_POWER`, logs the record to GCS and returns `predicted_AC_POWER`.
`GET /drift`	Loads the reference and current windows, runs the KS monitor, stores a report and returns the status and per-feature results.
`GET /drift-report`	Reads the accumulated drift CSV from GCS and returns its records.
`GET /production-data`	Lists and returns the JSON prediction records under the GCS `production/` prefix.

The Pydantic request schema limits `IRRADIATION` to the range 0–2.0 and `AMBIENT_TEMPERATURE` to −10–100. An invalid date format returns HTTP 400, while prediction or storage failures are logged and returned as HTTP 500.

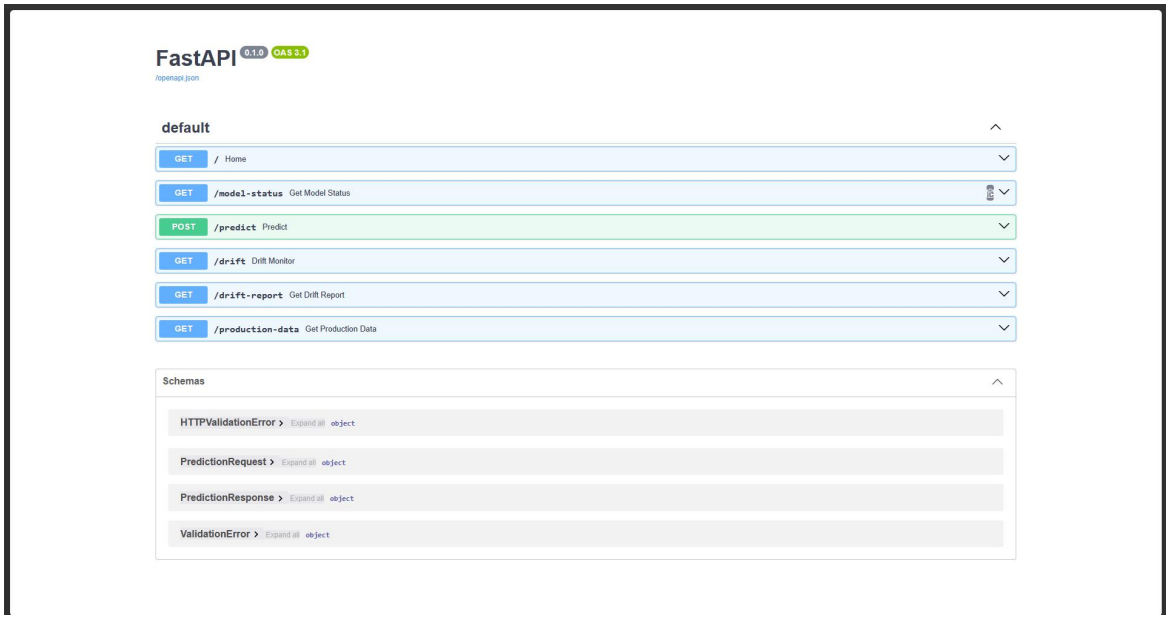


Figure 1. FastAPI Swagger interface served from Cloud Run, listing the six implemented endpoints and the request/response schemas.

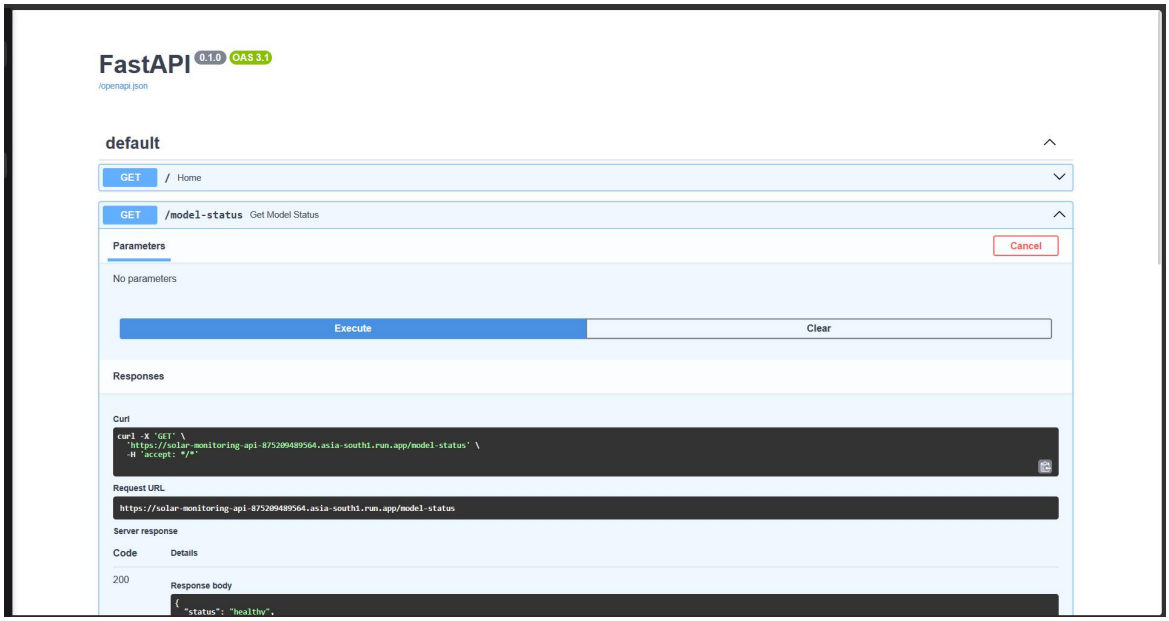


Figure 2. Response of GET /model-status on the deployed service (HTTP 200, status "healthy").

A test prediction was made with `IRRADIATION` = 0.65, `AMBIENT_TEMPERATURE` = 30.5 and `DATE_TIME` = "04-10-2026 14:00". The deployed service returned HTTP 200 with a predicted AC power of 18782.39 (Figure 3).

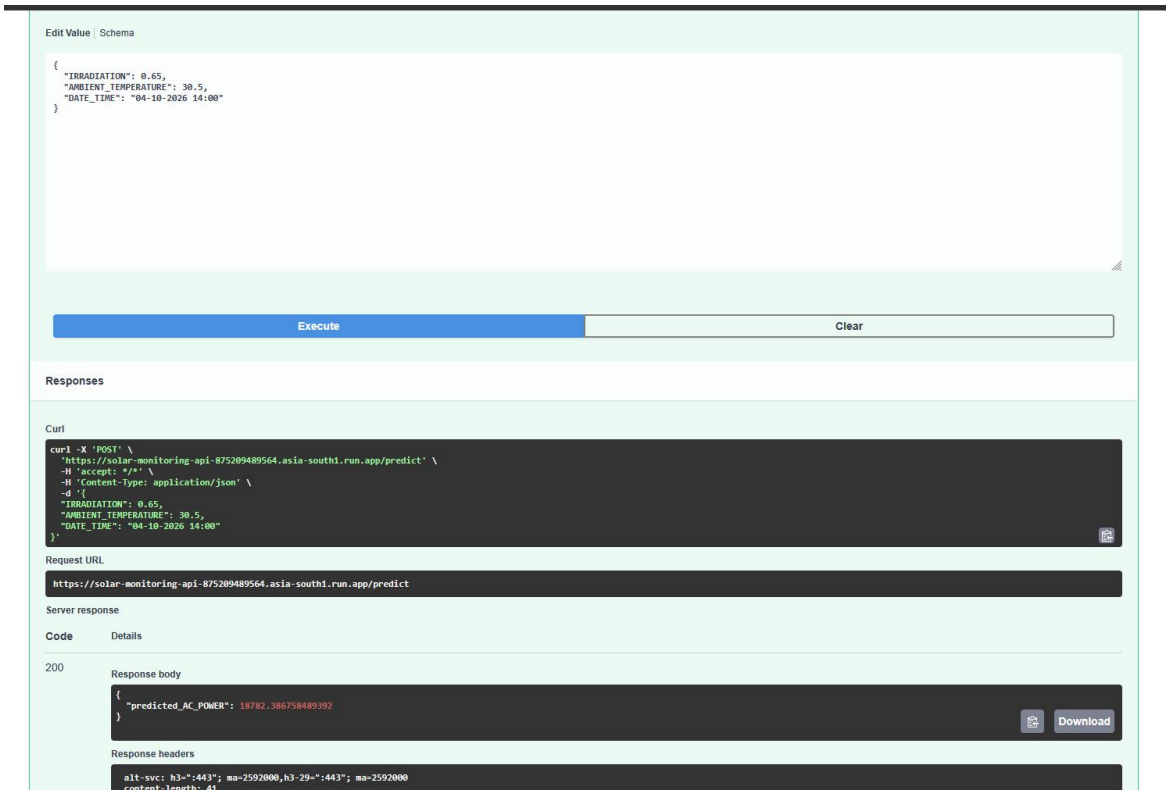


Figure 3. Successful POST /predict request and response on the Cloud Run URL.

8. Docker and Containerization

The Dockerfile uses `python:3.11-slim`, installs the pinned dependencies from `requirements.txt` (FastAPI, Uvicorn, Joblib, NumPy, Pandas, SciPy, scikit-learn, Google

Cloud Storage and Requests), copies the application into `/app` and starts Uvicorn on `0.0.0.0:8080`. The `.dockerignore` excludes local environment files, caches, virtual environments, reports, notebooks, backups and Git metadata, but keeps `data/` and `models/`, which the running application needs. Credentials are never copied into the image.

9. Google Cloud Deployment

The container image is built locally, pushed to a Docker repository in Artifact Registry and deployed to managed Cloud Run. Figure 5 shows the repository containing the pushed `solar-monitoring-api` image, and Figure 6 shows the running Cloud Run service. The project ID and region visible in these console captures are summarised below.

Resource	Value
Project ID	`solar-power-monitoring-511206`
Region	`asia-south1` (Mumbai)
Cloud Run service	`solar-monitoring-api`
Service URL	`https://solar-monitoring-api-875209489564.asia-south1.run.app`
Artifact Registry repository	`solar-monitoring` (Docker, Standard)
Container image	`solar-monitoring-api`
GCS bucket	`solar-monitoring-511206` (Standard, not public, soft delete)
Runtime service account	`solar-monitoring-run@solar-power-monitoring-511206.iam.gserviceaccount.com` (as documented)

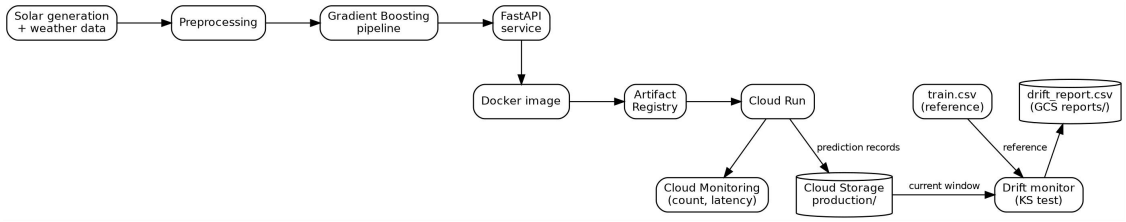


Figure 4. Overall system architecture: model serving on Cloud Run, prediction logging to Cloud Storage and the drift-monitoring path.

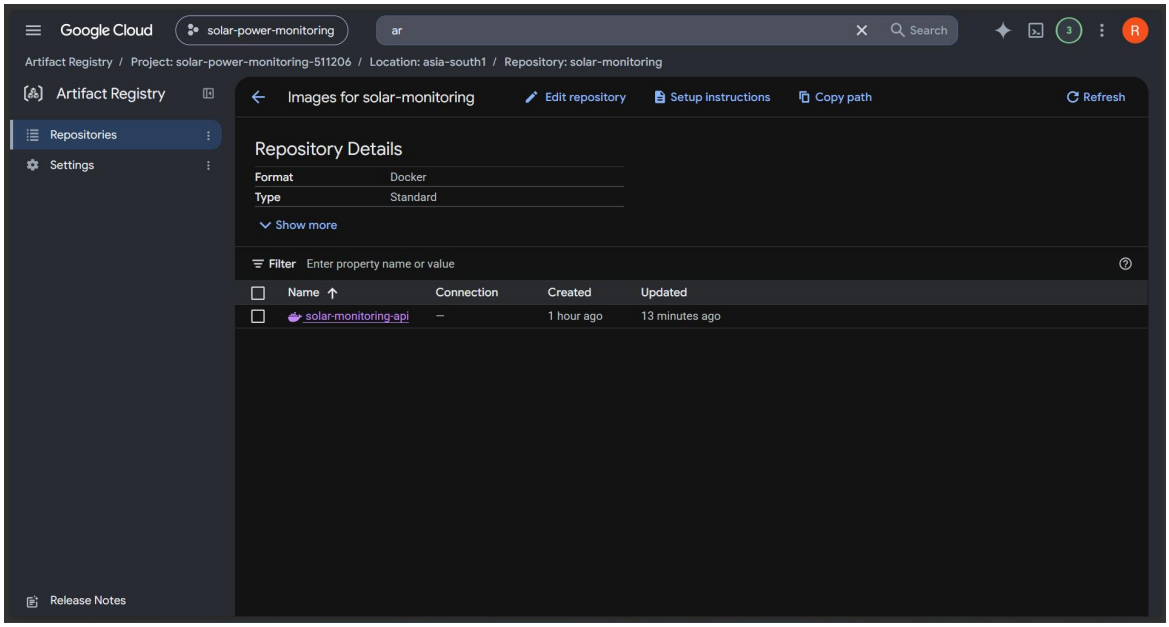


Figure 5. Artifact Registry repository solar-monitoring in project solar-power-monitoring-511206 (asia-south1) containing the solar-monitoring-api Docker image.

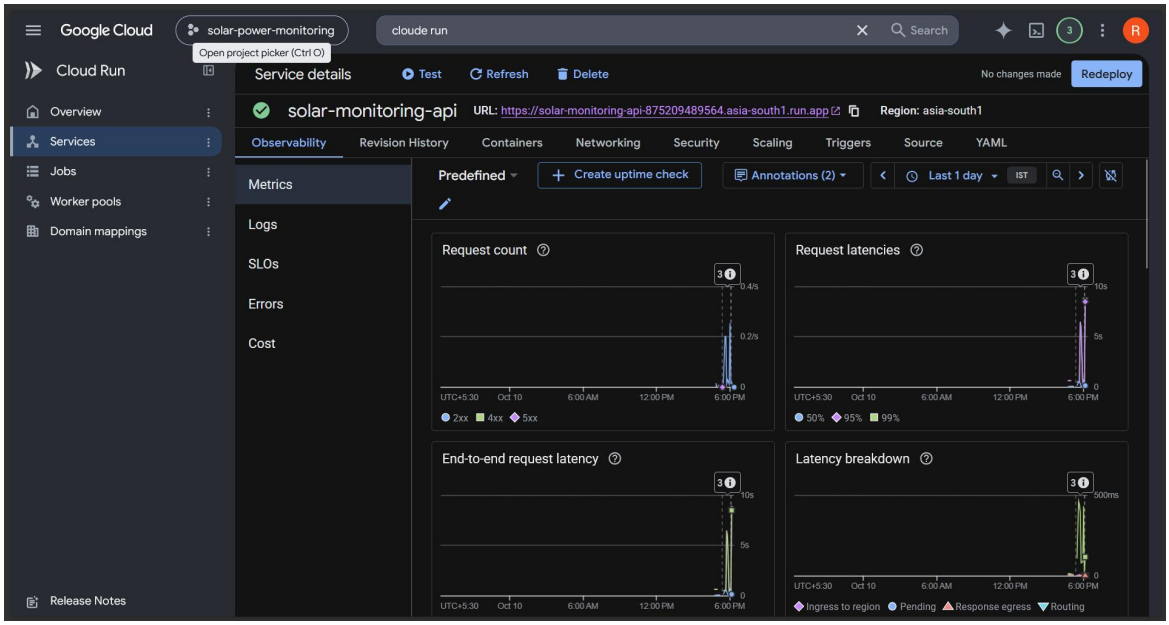


Figure 6. Cloud Run service solar-monitoring-api in asia-south1 with a healthy status, showing the service URL and the Observability/Metrics tab.

The deployment uses `--allow-unauthenticated`, so the API is publicly reachable. This is convenient for an academic demonstration but is addressed in Section 16. The repository contains a helper (`monitoring/monitoring_trigger.py`) that counts stored production objects and reports whether at least 50 new records have accumulated, but no Cloud Scheduler job was configured or evidenced in this project; drift checks were run by calling the `/drift` endpoint.

10. Production Prediction Logging

After each prediction, `utils/logger.py` creates a timestamp and a unique identifier and uploads a JSON object to `gs://solar-monitoring-511206/production/`. Each record

contains `timestamp`, `IRRADIATION`, `AMBIENT_TEMPERATURE`, `HOUR_SIN`, `HOUR_COS` and `predicted_AC_POWER`. Logging is synchronous and occurs before the response is returned, so client-observed latency includes the GCS upload in addition to prediction and network time. No ground-truth `AC_POWER` label is stored.

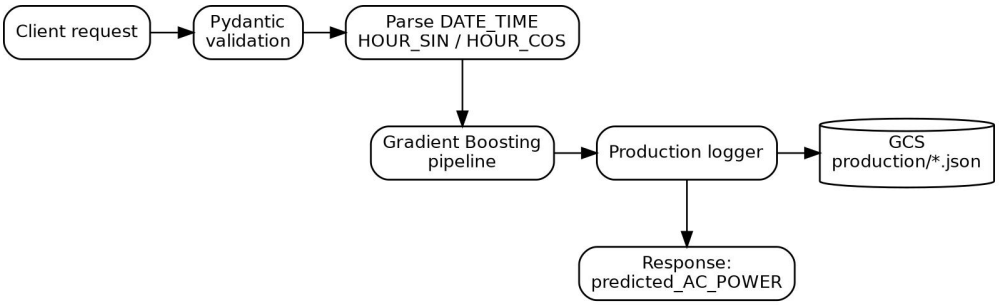


Figure 7. Prediction and production-data flow: a valid request is converted to four model features, the prediction is logged to GCS and then returned.

Figure 8 shows the `production/` folder in the bucket, with one JSON object per prediction (roughly 200–230 bytes each). Figure 9 shows the `/production-data` endpoint returning these records through the API: 122 records were available at the time of capture.

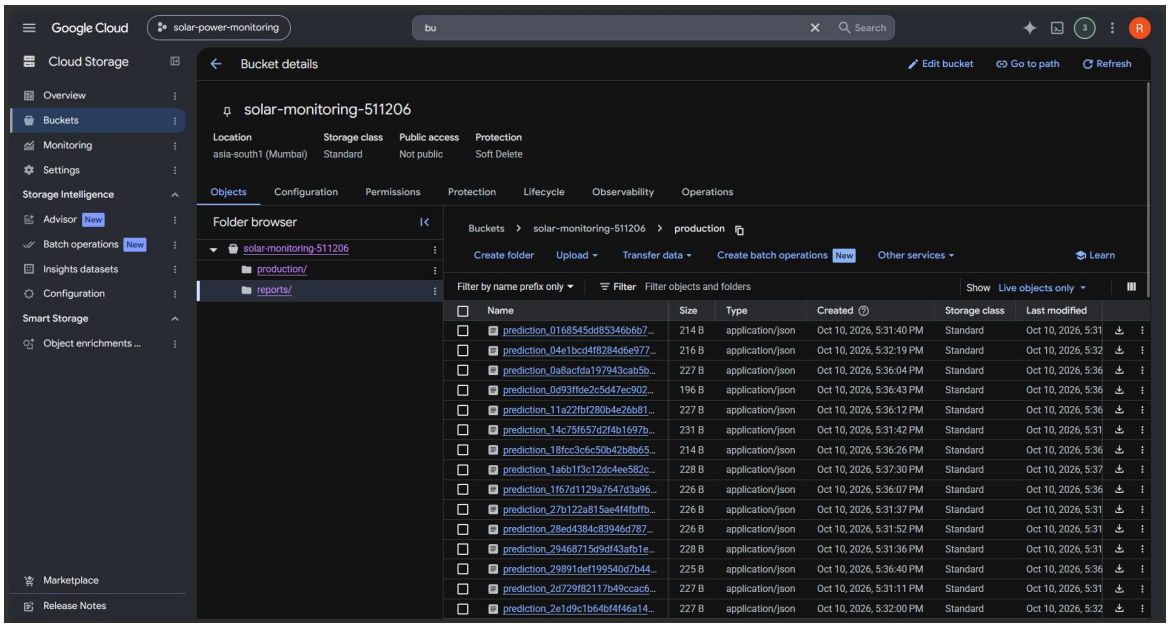


Figure 8. Prediction records stored as JSON objects in the production/ folder of the Cloud Storage bucket.

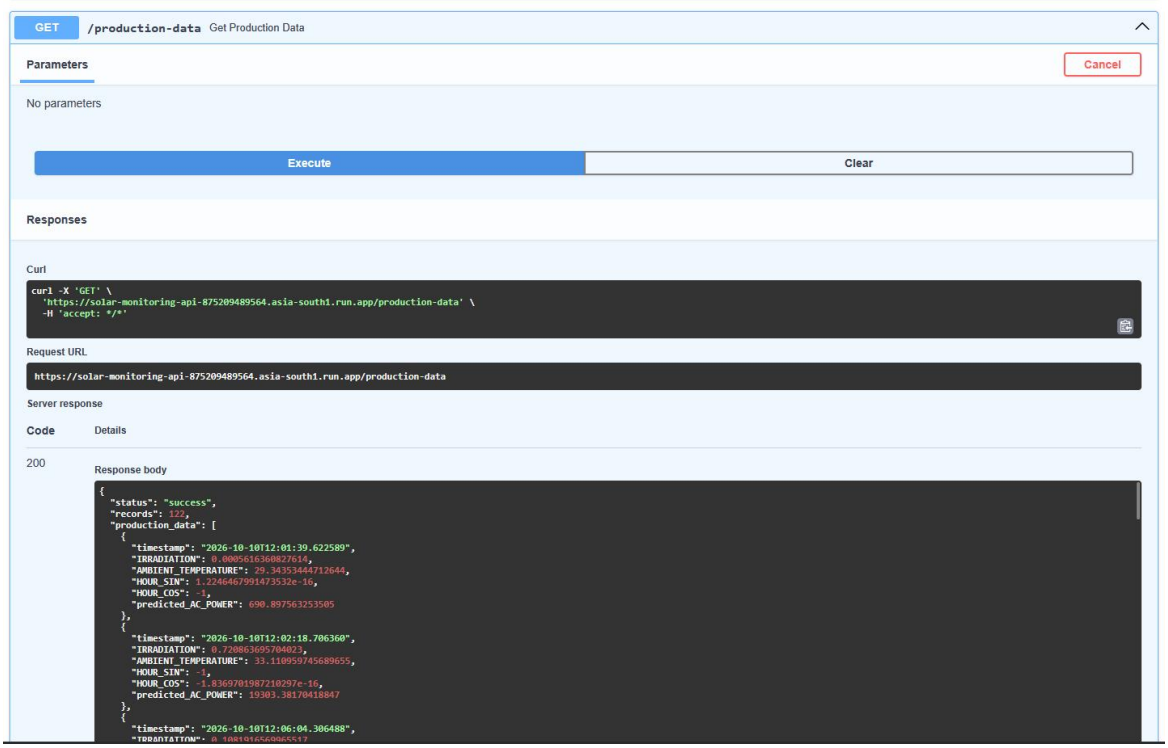


Figure 9. GET /production-data response: status “success” with 122 stored production records.

11. Drift Monitoring

11.1 Reference and current windows

- **Reference:** `data/train.csv` (Plant 1 training population, 469 rows).
- **Current:** JSON prediction records under the GCS `production/` prefix. The loader parses timestamps, sorts the records newest-first and keeps up to the reference-window size. At least 50 records are required before drift is evaluated.
- **Production population:** the generator script sends Plant 2 validation rows to the deployed API, which is the project’s domain-shift mechanism.

11.2 Statistical test

The monitor tests `IRRADIATION` and `AMBIENT_TEMPERATURE` with SciPy’s two-sample `ks_2samp`. For each feature it records the KS statistic, p-value, alpha (0.05), a Boolean drift flag and a monitoring timestamp. A feature is flagged when `p_value < alpha`. The overall `/drift` status is `DRIFT DETECTED` if any feature is flagged, `NO DRIFT` otherwise, and `NOT ENOUGH DATA` when fewer than 50 current records exist.

The KS statistic is the maximum distance between the empirical cumulative distribution functions of the two samples. If $p < 0.05$ the null hypothesis of equal distributions is rejected for that feature. If $p \geq 0.05$ there is insufficient evidence of drift, which is not proof that the distributions are identical. The threshold is applied independently to two features without multiple-comparison correction, and the test says nothing about model accuracy or whether retraining is needed.

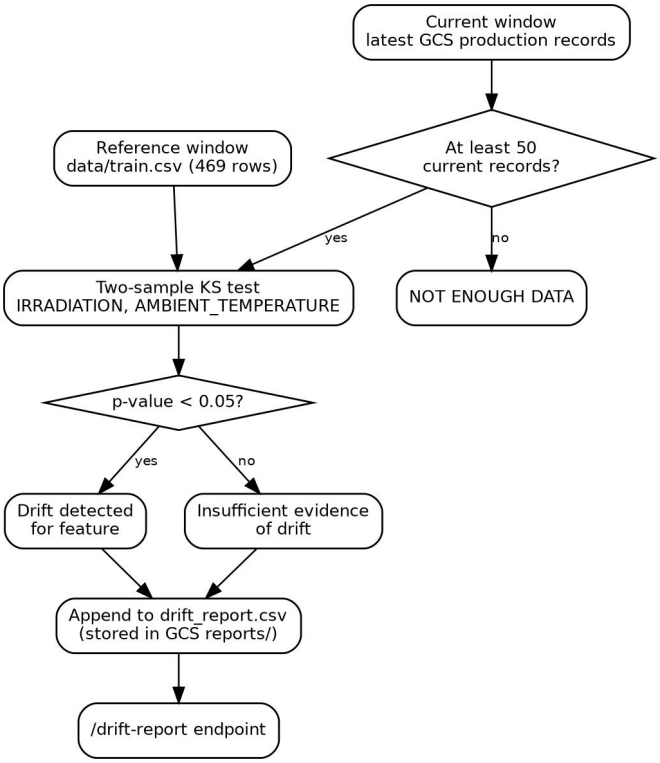


Figure 10. Drift-monitoring architecture: the training set is the reference, recent GCS predictions form the current window, and a 50-record minimum gates the KS comparison.

12. Drift Detection Results

Two drift runs are evidenced by the captured outputs. The first is a live call to `GET /drift`, which compared 122 current records against 469 reference records (Figure 11). The second is the report stored in the bucket and returned by `/drift-report` (Figures 12–14).

12.1 Live drift check (/drift endpoint)

Feature	KS statistic	p-value	Alpha	Drift detected
IRRADIATION	0.1591	0.01292	0.05	True
AMBIENT_TEMPERATURE	0.3968	3.44×10^{-14}	0.05	True

Overall status: **DRIFT DETECTED**. Monitoring time 2026-10-10T12:14:15 UTC; current records 122; reference records 469.

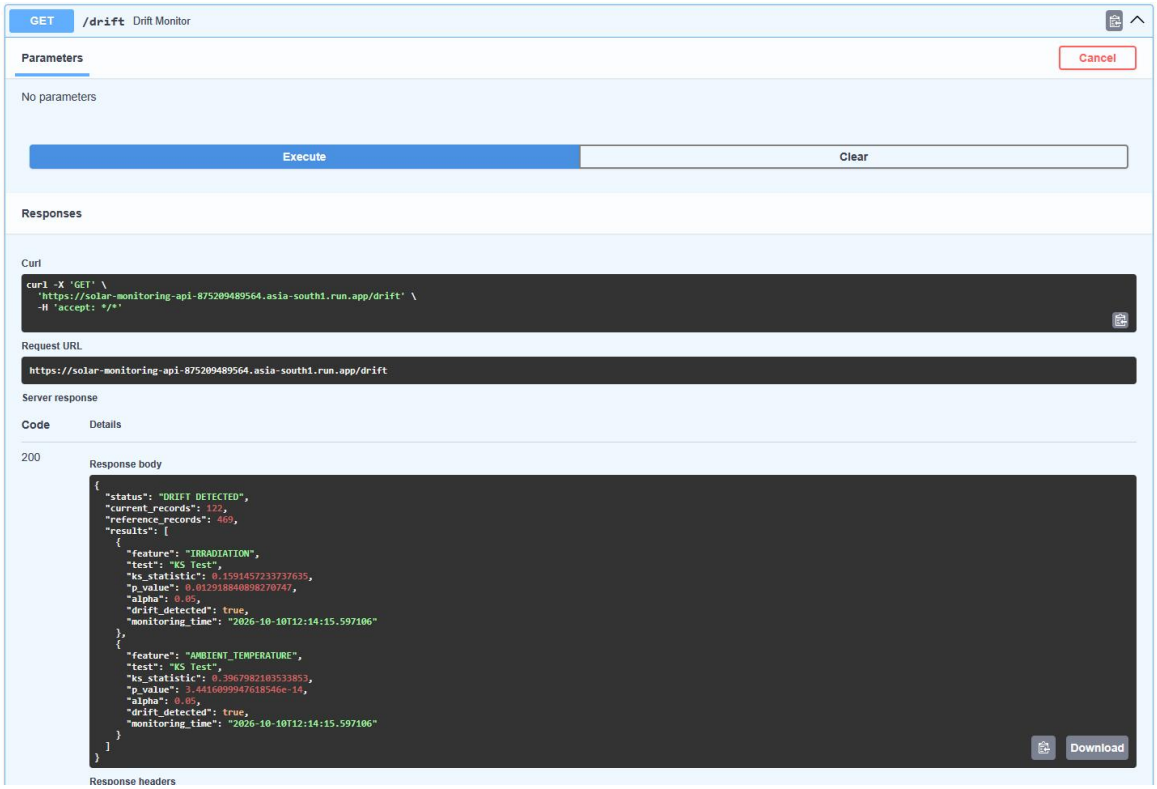


Figure 11. GET /drift response: status “DRIFT DETECTED”, 122 current and 469 reference records, with KS results for both features.

12.2 Stored drift report

The report is persisted as `reports/drift_report.csv` in the bucket (276 bytes, created 6:09:50 PM IST) and read back through `/drift-report`. Its timestamp (2026-10-10T12:39:49 UTC) corresponds to that upload, since IST is UTC+5:30.

Feature	KS statistic	p-value	Alpha	Drift detected
IRRADIATION	0.1604	0.002026	0.05	True
AMBIENT_TEMPERATURE	0.3945	8.91×10^{-19}	0.05	True

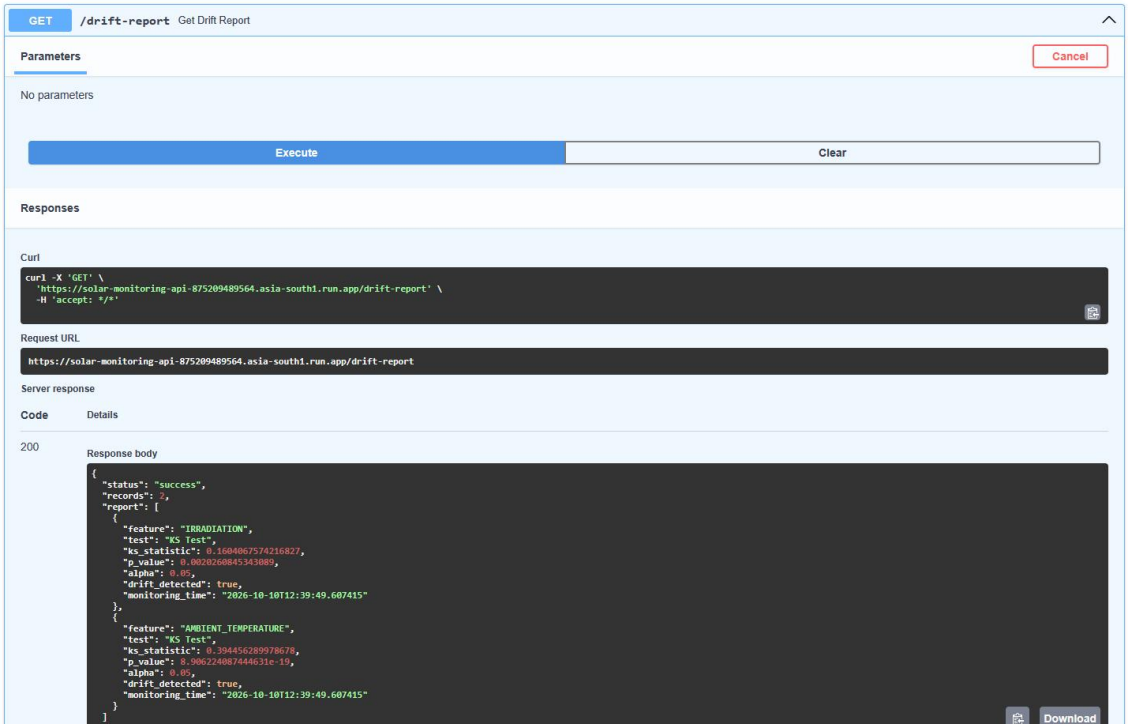


Figure 12. GET /drift-report response returning the stored report (status “success”, 2 records).

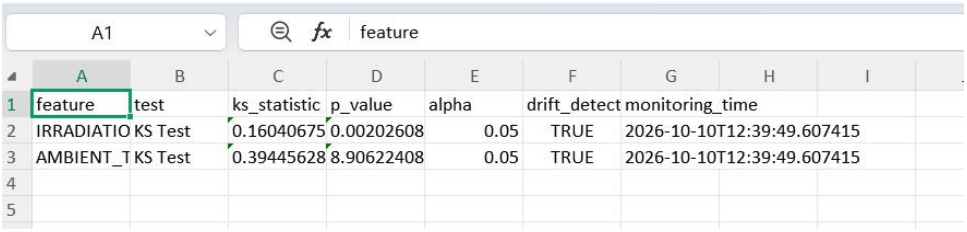


Figure 13. Stored drift_report.csv opened in a spreadsheet: one row per tested feature.

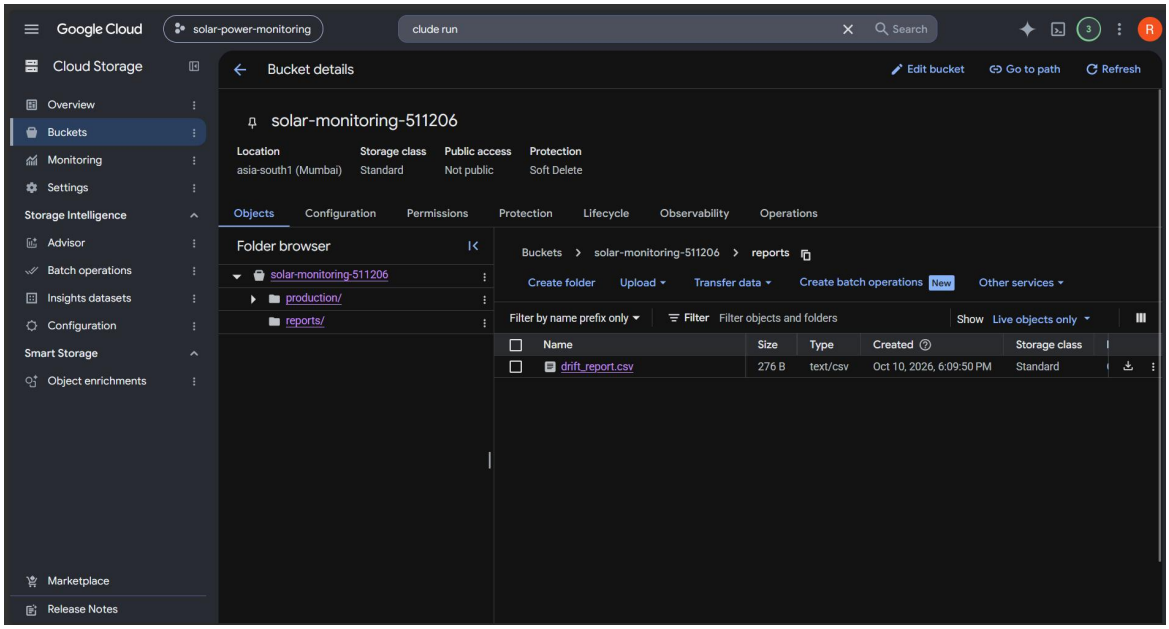


Figure 14. drift_report.csv stored in the reports/ folder of the Cloud Storage bucket.

12.3 Interpretation

In both runs ambient temperature shows a large separation from the reference (KS ≈ 0.39–0.40, p far below 0.05), and irradiation shows a smaller but still significant separation (KS ≈ 0.16, $p < 0.05$). The two runs give slightly different statistics because the current window changed as more production records accumulated between them; the report does not store sample sizes, so the exact windows cannot be reconstructed from it (see Section 17). The KS statistic for ambient temperature means the empirical CDFs differ by at most about 0.40 — a distance measure, not a percentage of prediction error.

The result is consistent with the Plant 1 versus Plant 2 domain shift and justifies investigating data ranges and quality. It does not by itself prove model degradation or justify automatic retraining; that requires true outcomes against which prediction error can be measured.

13. Latency Monitoring and Load Testing

`scripts/load_test.py` targets the Cloud Run `/predict` URL with a fixed valid payload and a `ThreadPoolExecutor` of 10 workers submitting 100 requests. For every response it measures elapsed client-side time with `time.perf_counter`, counts HTTP 200 responses as successful and computes p50, p95 and p99 by linear interpolation over the successful latencies. Failed requests are counted but excluded from the percentiles, and a 30-second timeout applies.

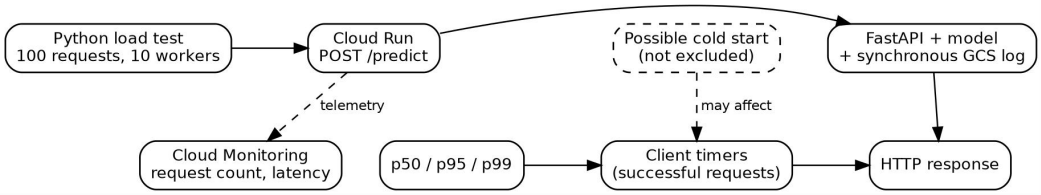


Figure 15. Latency monitoring and load-testing flow: the client script measures round trips, while Cloud Monitoring separately records Cloud Run service telemetry.

13.1 Load-test result

Measurement	Value
Total requests	100
Concurrency	10
Successful / failed requests	100 / 0
p50 latency	2189.88 ms
p95 latency	3277.93 ms
p99 latency	3472.61 ms
Total test duration	22.98 s

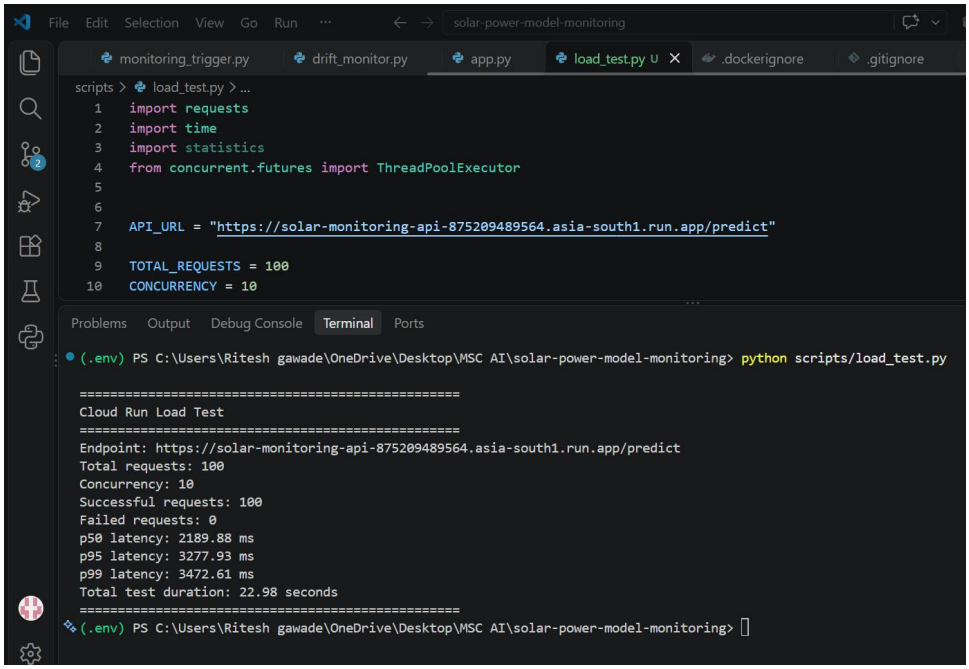


Figure 16. Terminal output of the controlled load test: 100 requests, concurrency 10, 100 successful, 0 failed, with p50/p95/p99 latency.

13.2 Cloud Monitoring percentiles

The Metrics Explorer query used the "Cloud Run Revision – Request Latency" metric with 50th, 95th and 99th percentile aggregations, grouped by `service_name`, over the window 9 Oct 5:38 PM – 10 Oct 5:38 PM (IST).

Percentile	Cloud Monitoring value
p50	462.022 ms
p95	534.059 ms
p99	544.921 ms

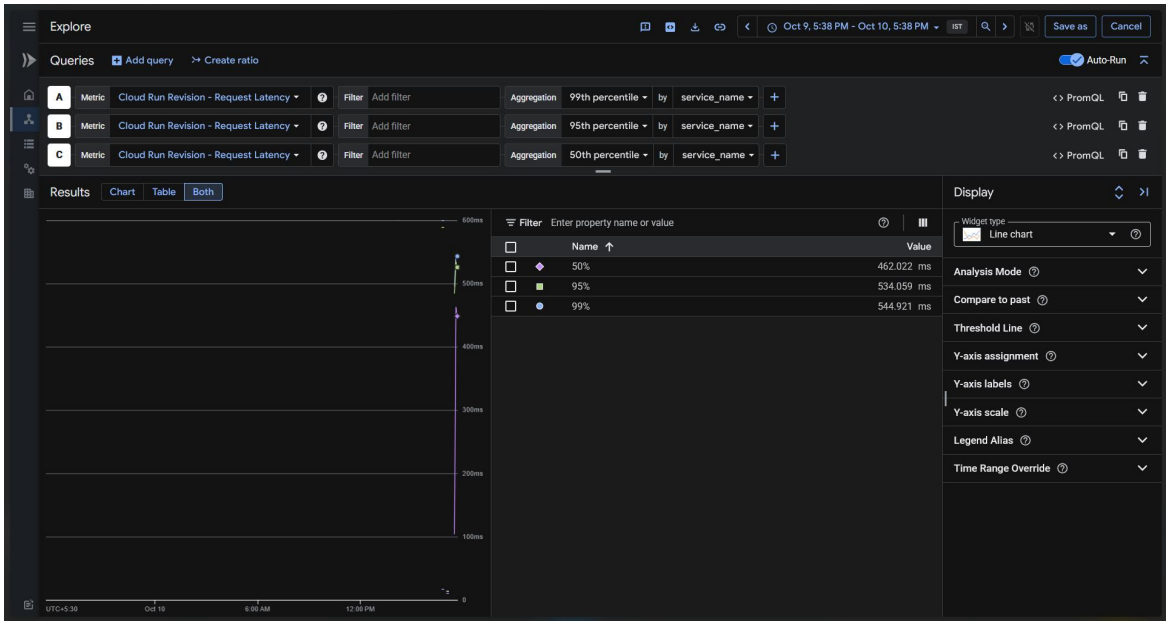


Figure 17. Cloud Monitoring Metrics Explorer: Cloud Run request latency at the 50th, 95th and 99th percentiles.

13.3 Why the two sets of numbers differ

The two sets of values should not be equated. The Cloud Monitoring window ended at 5:38 PM, whereas the load test ran at around 6:00 PM (the request-count peak in Figure 18 and the 6:09 PM report upload both fall after the window). The Explorer values therefore describe the earlier traffic produced while the production records were being generated (objects in Figure 8 were created between about 5:31 and 5:37 PM). In addition, the load-test numbers are measured at the client and include internet round-trip time, TLS, request queueing under ten simultaneous requests, and the synchronous GCS write, whereas Cloud Monitoring reports latency observed by Cloud Run. Cold-start delay after inactivity can also affect the first requests; the script neither identifies nor excludes cold starts. These explanations are plausible but were not separately measured.

Percentiles are used because averages hide tail behavior: p50 describes typical response time, p95 shows the slower tail and p99 the extreme tail. For this service the load-test tail (p99 ≈ 3.5 s) is about 1.6 times its median, indicating relatively consistent behavior under the stated load, and no request failed.

14. Cloud Monitoring

Cloud Run exposes built-in service metrics without extra instrumentation. The **Request count** chart, split by response-code class, shows the traffic generated during the experiment: the 2xx series peaks at about 0.253 requests per second at 6:00 PM on 10 Oct 2026, with no 4xx or 5xx responses recorded (Figure 18). The Cloud Run service page also shows request latency, end-to-end latency and latency-breakdown charts, which spike at the same time as the load test.

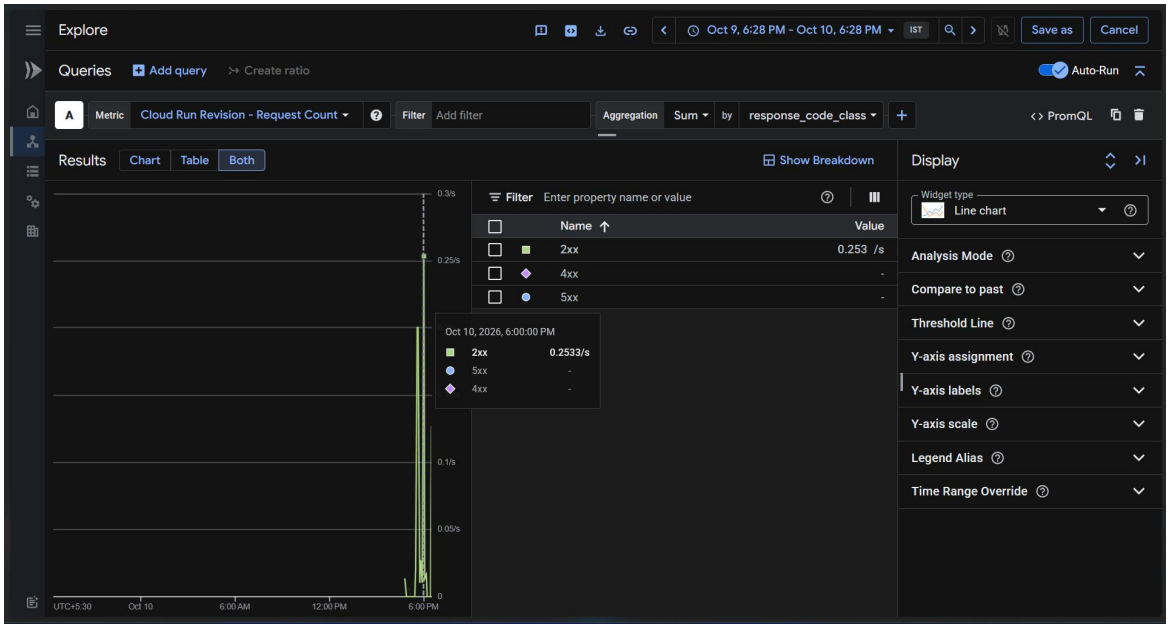


Figure 18. Cloud Monitoring Metrics Explorer: Cloud Run request count by response-code class. Only 2xx responses are present; the peak is 0.2533 requests/s.

The application-level `/drift` endpoint and the GCS drift report provide separate model-input monitoring evidence. They are not Cloud Monitoring alert policies. No alerting policy or Cloud Scheduler job was created in this project.

15. Results and Discussion

- **Model quality (offline):** Gradient Boosting reached RMSE 523.10 and R^2 0.9964 on the held-out split, and RMSE 627.61 / R^2 0.9937 under 5-fold cross-validation. This does not substitute for evaluation on labeled production outcomes.
- **Drift:** both runs flagged ambient temperature strongly and irradiation more weakly. The Plant 1 to Plant 2 change is a reasonable cause to investigate, but drift does not directly measure accuracy, label shift, concept drift or prediction drift.
- **Latency:** under 100 requests at concurrency 10 the service handled all requests without failure; median client latency was about 2.2 s and p99 about 3.5 s, against server-side Cloud Run percentiles below 550 ms for the earlier traffic window.
- **Reliability:** Cloud Monitoring recorded only 2xx responses during the experiment.

The practical follow-up to the drift result is to inspect the current and reference populations, verify feature ranges and data quality, and obtain true outcomes before deciding on retraining. For latency, the synchronous GCS write inside `/predict` is the most obvious candidate for optimization (for example by logging asynchronously), together with a review of minimum instances to limit cold starts.

16. Security and Cost Management

16.1 Security

The Cloud Run service is deployed unauthenticated, which exposes the API publicly. A production deployment should add authentication, abuse protection and access controls. A dedicated runtime service account is used for GCS access (as documented), the bucket is not publicly accessible, and no credentials, tokens or key files are included in the image, repository description or this report. `.env`, virtual environments, backups, caches and Git metadata are excluded from the build.

16.2 Cost control

Two monthly budgets were created in the “Billing Account for Education”: a **$10 Monthly Budget Alert** (alerts at 50%, 90%, 100% and 150%) and a **Solar Monitoring $5 Budget Alert** (alerts at 50%, 90% and 100%). The console shows the second budget being created and both budgets at $0.00 spend with no credits used. A budget alert is only a notification, not a hard spending cap, so billing should still be reviewed regularly, and unneeded resources (Cloud Run services, images, buckets) should be removed once the assignment is complete.

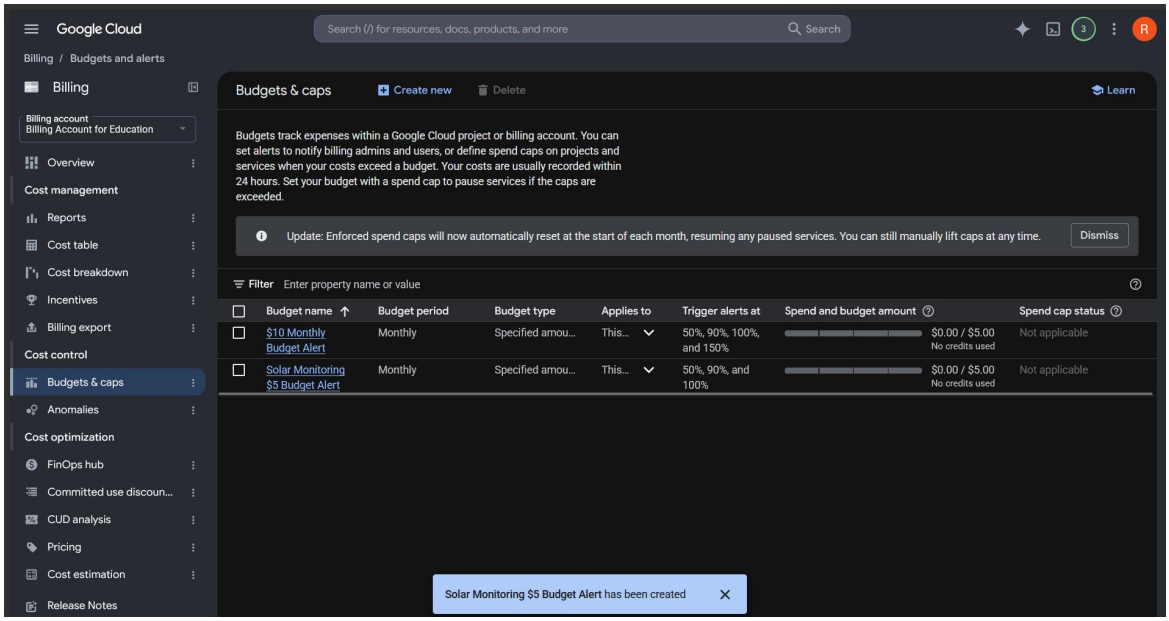


Figure 19. Billing "Budgets & caps" page showing the $10 monthly and $5 budget alerts with their alert thresholds.

17. Limitations

1. Production records are generated from validation data rather than a long-running stream of natural plant telemetry.
2. Only two numeric input features are tested; prediction drift, label shift and concept drift are not monitored.
3. Production records carry no true `AC_POWER`, so online error and quality monitoring is not possible.
4. The drift report does not store sample sizes or record identifiers, limiting independent reconstruction of each run.
5. The KS tests are applied to two features without multiple-comparison correction.
6. Cold starts are neither detected nor excluded in the load test, and the Cloud Monitoring window and load-test window do not overlap exactly.
7. No Cloud Scheduler job or Cloud Monitoring alerting policy was configured; drift checks are triggered manually.
8. Unauthenticated access is suitable only for a controlled demonstration.

18. Future Improvements

- Export labeled production outcomes and compute time-windowed RMSE, MAE and R^2 to see whether drift corresponds to quality loss.
- Monitor prediction distributions and add further tests (for example PSI or Chi-square for categorical fields).
- Record reference/current window sizes, boundaries and sample identifiers with every drift run.

- Add a Cloud Scheduler job that invokes `/drift`, and a Cloud Monitoring alerting policy once its behavior is tested.
- Log predictions asynchronously and set a minimum instance count to reduce latency and cold starts.
- Restrict API access and apply least-privilege IAM before any wider deployment.
- Build a dashboard linking request count, latency percentiles and drift results, and define a review process for retraining decisions.

19. Conclusion

The project delivers an end-to-end monitored ML deployment: a trained regressor served by FastAPI, containerized with Docker, stored in Artifact Registry and run on Cloud Run, with every prediction logged to Cloud Storage. A KS-based drift monitor compared 122 production records with the 469-row reference and flagged both irradiation and ambient temperature at alpha 0.05, with ambient temperature showing by far the larger shift. A 100-request, 10-concurrent load test completed with zero failures, giving client-side p50/p95/p99 of 2189.88 / 3277.93 / 3472.61 ms, while Cloud Monitoring showed lower server-side percentiles and only 2xx responses. Budget alerts kept the work cost-controlled.

The drift findings indicate a distributional change that warrants investigation, but they cannot establish degraded model quality without true outcomes. Automated scheduling and alerting are the main steps that remain for a production-ready monitoring setup.

20. Assignment Requirement / Compliance Table

Requirement	Implementation	Evidence	Status
Own model / API	Gradient Boosting pipeline served by FastAPI; `/predict` returns `predicted_AC_POWER`.	Figures 1–3 (Swagger UI, model-status, predict).	Met
Google Cloud deployment	Docker image in Artifact Registry, deployed on Cloud Run (`asia-south1`).	Figures 5, 6.	Met
Cloud monitoring	Cloud Run request count and latency metrics in Cloud Monitoring.	Figures 17, 18.	Met
p50 / p95 latency under stated load	100 requests, 10 concurrent; p50 2189.88 ms, p95 3277.93 ms, p99 3472.61 ms.	Figure 16.	Met
Reference vs current drift with statistical test	KS test (alpha 0.05), 469 reference vs 122 current records.	Figures 11–14.	Met
Interpret drift without equating it to degradation	Discussion in Sections 12.3 and 15.	Text; no labels logged.	Met
Project ID screenshot	Project `solar-power-monitoring-511206`.	Figures 5, 6 (project selector / breadcrumb).	Met
Budget alert	$5 and $10 monthly budgets with threshold alerts.	Figure 19.	Met
Cloud Scheduler automation	Helper checks for 50 new records; no scheduled job created.	`monitoring_trigger.py` only.	Not done
Credential safety	No secrets in report or image; ignore files exclude `.env`.	`.gitignore`, `.dockerignore`.	Met

21. Figure / Evidence Index

Figure	Content	Report section
1	FastAPI Swagger UI on Cloud Run	7
2	/model-status response	7
3	/predict request and response	7
4	System architecture diagram	9
5	Artifact Registry image	9
6	Cloud Run service details	9
7	Prediction and production-data flow	10
8	Prediction records in GCS	10
9	/production-data response	10
10	Drift-monitoring architecture	11

Figure	Content	Report section
11	/drift response	12
12	/drift-report response	12
13	Drift report CSV	12
14	Drift report in GCS bucket	12
15	Latency-monitoring and load-test flow	13
16	Load-test terminal output	13
17	Cloud Monitoring latency percentiles	13
18	Cloud Monitoring request count	14
19	Budget alerts	16

22. Commands Used

22.1 Build, tag, authenticate and push the container

```
docker build -t solar-monitoring-api .

gcloud auth configure-docker asia-south1-docker.pkg.dev

docker tag solar-monitoring-api:latest \
  asia-south1-docker.pkg.dev/solar-power-monitoring-511206/solar-monitoring/solar-
monitoring-api:latest

docker push asia-south1-docker.pkg.dev/solar-power-monitoring-511206/solar-
monitoring/solar-monitoring-api:latest
```

22.2 Deploy to Cloud Run

```
gcloud run deploy solar-monitoring-api \
  --image=asia-south1-docker.pkg.dev/solar-power-monitoring-511206/solar-
monitoring/solar-monitoring-api:latest \
  --region=asia-south1 \
  --platform=managed \
  --allow-unauthenticated \
  --service-account=solar-monitoring-run@solar-power-monitoring-
511206.iam.gserviceaccount.com
```

22.3 Project scripts

```
python scripts/generate_production_data.py
python scripts/load_test.py
python monitoring/monitoring_trigger.py

# local authentication (Application Default Credentials)
gcloud auth application-default login
```

The production-data script sends up to 60 validation records per run to the prediction URL; the load-test script sends the separate 100-request/10-concurrency test. The trigger script only checks

the stored object count and does not itself call `/drift`. Local ADC files and service-account keys must not be included in a submitted archive.

23. Project Structure

```
solar-power-model-monitoring/
├── app.py
├── Dockerfile
├── .dockerignore
├── .gitignore
├── requirements.txt
├── data/
│   ├── train.csv
│   ├── validation.csv
│   ├── validation_drift.csv
│   └── production_predictions.csv
├── models/
│   └── model_pipeline.joblib
├── monitoring/
│   ├── drift_monitor.py
│   └── monitoring_trigger.py
├── notebooks/
│   └── 01_dataset_inspection_ipynb.ipynb
├── reports/
│   └── drift_report.csv
├── schema/
│   ├── prediction_request.py
│   ├── prediction_response.py
│   └── __init__.py
├── scripts/
│   ├── generate_production_data.py
│   └── load_test.py
└── utils/
    ├── data_loader.py
    ├── features.py
    ├── logger.py
    └── __init__.py
```

`data/production_predictions.csv` is a local artifact and is not read by the drift endpoint; the active monitor reads JSON objects from the GCS `production/` prefix. Credentials, virtual environments, caches, Git metadata and `backup/` are omitted.